



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251184
Nama Lengkap	Matthew Jason Pratama
Minggu ke / Materi	15 / Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI

**UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Pengertian Rekursif

Fungsi rekursif adalah fungsi yang berisi dirinya sendiri, jadi fungsi ini memanggil dirinya sendiri. Fungsi rekursif merupakan fungsi matematis yang berulang. Namun biasanya fungsi ini perlu diperhatikan agar fungsinya berhenti dan tidak menghabiskan memori. Fungsi rekursif merupakan fungsi yang harus digunakan hati hati agar tidak menghasilkan infinite loop.

Karena fungsi rekursif akan terus berjalan sampai kondisi berhenti atau base case terpenuhi. Jika kondisi berhenti tidak dibuat dengan benar, fungsi akan terus berjalan tanpa batas (infinite loop) akan menyebabkan program hang up atau stack overflow.

Cara pemanggilan fungsi ada 3 cara:

- Berasal dari bagian utama program
- Dari fungsi yang lainnya
- Dari fungsi itu sendiri

Base Case adalah bagian penentu untuk rekursif berhenti

Rekursif Case adalah bagian dimana statement yang akan diulang terus menerus

Kelebihan dan Kekurangan

Ada beberapa kekurangan dan kelebihan ketika menggunakan rekursif.

Kelebihan:

1. Kode singkat
2. Masalah kompleks dapat di breakdown menjadi sub masalah yang lebih kecil di dalam rekursif

Kekurangan:

1. Memakan memori yang lebih banyak karena setiap bagian dipanggil maka membutuhkan sejumlah ruang memori.
2. Mengorbankan efisiensi dan kecepatan.
3. Sulit dilakukan debugging dan kadang sulit dimengerti.

Beberapa cara untuk mengoptimisasi fungsi rekursif:

1. Memoization: hasil pemanggilan rekursif sebelumnya disimpan dalam cache sehingga tidak perlu dihitung ulang.
2. Tail recursion: usahakan membentuk menjadi tail recursion agar lebih efisien.
3. Minimalisasi kedalaman rekursif.

Bentuk Umum dan Studi Kasus

Bentuk umum dari rekursif yaitu

Def namafungsi(parameter):

...

namafungsi(parameter)

Jenis jenis rekursif

Head Recursion: pemanggilan rekursif di bagian awal fungsi. Proses dilakukan setelah rekursif kembali.

Tail Recursion: Pemanggilan rekursif di bagian akhir fungsi. Proses dilakukan sebelum rekursi.

Indirect Recursion: Melibatkan lebih dari satu fungsi. Kurang direkomendasikan.

Linear Recursion: Hanya memanggil satu rekursif dalam satu pemanggilan.

Tree Recursion: Memanggil dua atau lebih rekursif sekaligus. Contohnya: fibonacci

Studi kasus rekursif

- Faktorial:

Faktorial adalah perkalian deret angka dari 1 hingga n. Ditulis dengan n!. Contoh: $4! = 4 \times 3 \times 2 \times 1$

Kode rekursif faktorial:

Base case: $n(0) = 1$ atau $n(1) = 1$

Def factorial(n)

 If $n == 0$ or $n == 1$:

 Return 1

 Else:

 Return factorial(n-1) * n

print(factorial(4))

Proses perhitungannya

factorial(4) > 4 * factorial(3)

factorial(3) > 3 * factorial(2)

factorial(2) > 2 * factorial(1)

factorial(1) > 1

- Fibonacci:

Deret fibonacci adalah barisan yang bilangannya berupa penjumlahan dari 2 bilangan sebelumnya. Contoh: 1, 1, 2, 3, 5, 8, 13, 21,

Rumus rekursif dari fibonacci:

Base case: Fibo(1) = 1 dan Fibo(2) = 1

Fungsi ini termasuk tree recursion karena memanggil 2 fungsi rekursif sekaligus

Def Fibo(n):

 If n == 1 or n == 2:

 Return 1

 Else:

 Return Fibo(n-1)+Fibo(n-2)

print(Fibo(4))

Proses perhitungannya:

Fibo(4) > Fibo(3) + Fibo(2)

Fibo(3) > Fibo (2) + Fibo(1)

Fibo(2) > 1

Fibo(1) > 1

Hasilnya 1 + 1 + 1

- Permutasi

Permutasi adalah peraturan sejumlah objek dari kumpulan angka.

Rumus: $P(n,r) = n! / (n-r)!$

Pola rekursifnya:

Base case m == 0 return 1

Def permutasi(n,m):

 If n ==0:

 Return 1

 Else:

 Return permutasi(n, m-1) * (m-n+1)

- Tower of Hanoi

Tower of hanoi merupakan masalah klasik rekursif: memindahkan keping sebanyak n dari tiang A ke tiang C menggunakan tiang B sebagai bantuan.

Strateginya pindahkan keping n-1 ke transit, kemudian pindahkan keping terbesar ke tujuan
lalu pindahkan keping n-1 ke tujuan

```
Def towehanoi(n,asal,tujuan,transit):
```

```
    If n == 1:
```

```
        Return
```

```
    towerhanoi(n-1,asal,transit,tujuan)
```

```
    print(f"Pindah keping {n} dari {asal} ke {tujuan}")
```

```
    towerhanoi(n-1,transit,tujuan,asal)
```

```
towerhanoi(3,'A','C','B')
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

git:https://github.com/Jason-prat/71251184_matthew

SOAL 1

Source:

```
def primas(n,x=1,prima=0):  
    if x>n:  
        return prima  
    if n%x==0:  
        prima+=1  
    return primas(n,x+1,prima)  
n=int(input("Masukkan Bilangan: "))  
p=primas(n)  
if p ==2:  
    print(f"Bilangan {n} adalah Prima")  
else:  
    print(f"Bilangan {n} bukan bilangan Prima")
```

Output:

```
Masukkan Bilangan: 5  
Bilangan 5 adalah Prima  
PS C:\Users\ASUS\Documents\  
Masukkan Bilangan: 3  
Bilangan 3 adalah Prima
```

Penjelasan:

Program akan mengecek apakah bilangan yang dimasukkan user merupakan bilangan prima atau bukan. Pertama program akan meminta user memasukkan bilangan, kemudian masuk ke dalam fungsi dengan parameter n,x=1,dan prima=0. Jika nilai $x > n$ maka akan mengembalikan

prima, jika tidak akan dicek apakah $n \% x == 0$ jika ya variabel prima nilainya bertambah 1 jika tidak maka akan ke rekursif dengan nilai x ditambah 1 begitu terus karena jika bilangan bukan prima maka nanti nilai variabel prima pasti tidak mungkin 2 karena bilangan prima hanya bisa habis dibagi oleh 2 bilangan yaitu bilangan 1 dan bilangan dirinya sendiri, sedangkan bilangan bukan prima bisa dibagi oleh 2 lebih bilangan atau hanya 1 bilangan seperti angka 1 ia hanya bisa dibagi oleh 1 sehingga bukan bilangan prima.

SOAL 2

Source:

```
def palindrom(kata):  
    if len(kata) <=1:  
        return "Kata Palindrome"  
    if kata[0] != kata[-1]:  
        return "Kata bukan Palindrome"  
    return palindrom(kata[1:-1])  
kata=input("Masukkan Kata: ")  
print(palindrom(kata))
```

Output:

```
Masukkan Kata: madam  
Kata Palindrome  
PS C:\Users\ASUS\Documents  
Masukkan Kata: katak  
Kata Palindrome  
PS C:\Users\ASUS\Documents  
Masukkan Kata: kataak  
Kata bukan Palindrome
```

Penjelasan:

Program berfungsi untuk mengecek apakah input dari user merupakan palindrome. Pertama program meminta user memasukkan kata kemudian dimasukkan ke dalam fungsi palindrom di dalam fungsi palindrom kata akan dicek apakah memiliki panjang kurang dari sama dengan 1

atau tidak karena jika kata hanya 1 huruf ia pasti palindrome dan akan mengembalikan "Kata Palindrome", jika tidak maka akan dicek apakah huruf pertama dan akhir kata beda jika ya maka kata tidak mungkin palindrome, jika sama maka akan lanjut ke rekursif, fungsi palindrom dipanggil dengan variabel kata tetapi hanya diambil dari indeks 1-(-1) agar menghilangkan huruf awal dan akhirnya, dan akan berulang terus sampai huruf tersisa 1 dan mengembalikan nilai "Kata Palindrome".

SOAL 3

Source:

```
def deret_ganjil(n,x=1,m=0,h=0):  
    x=2**m-1  
    h+=x  
    if m==n:  
        return h  
    else:  
        return deret_ganjil(n,x,m+1,h)  
  
n=int(input("Masukkan berapa kali perulangan: "))  
  
print(deret_ganjil(n))
```

Output:

```
Masukkan berapa kali perulangan: 4  
26  
PS C:\Users\ASUS\Documents\File Jasc  
Masukkan berapa kali perulangan: 5  
57
```

Penjelasan:

Program berfungsi untuk menjumlahkan deret bilangan ganjil dengan rumus $2^n - 1$ jadi akan membentuk pola $1+3+7+15+31+2^n-1$. Pertama program akan meminta user memasukkan

berapa jumlah n kemudian akan dimasukkan ke dalam fungsi `deret_ganjil`. Di dalamnya nanti akan disimpan nilai x yang merupakan urutan dengan nilai $2^m - 1$ dimana m merupakan urutan n yang kemudian akan ditambahkan ke variabel h yang menjadi nilai akhir, jika nilai variabel $m == n$ maka fungsi akan berhenti dan mengembalikan nilai h , jika tidak maka akan lanjut rekursif `deret_ganjil` dengan parameter $n, x, m+1, h$ sampai nilai m sama dengan n .

SOAL 4

Source:

```
def digit(angka):  
    if angka == 0:  
        return 0  
    return angka%10 + digit(angka//10)  
angka=int(input("Masukkan angka: "))  
print(digit(angka))
```

Output:

```
Masukkan angka: 234  
9  
PS C:\Users\ASUS\Docu  
Masukkan angka: 678  
21
```

Penjelasan:

Program akan menghitung jumlah masing masing digit yang dimasukkan oleh user. Pertama program akan meminta user memasukkan angka, kemudian dimasukkan ke dalam fungsi `digit`, di dalamnya dicek apakah angka bernilai 0 jika ya akan mengembalikan nilai 0, jika tidak akan fungsi akan mengembalikan nilai `angka%10` (diambil digit paling belakang) dan ditambah dengan rekursif `digit` dengan parameter `angka//10` jadi nanti akan dibuang digit paling belakang dan menyisakan digit puluhan ke atas dan berulang terus sampai nanti rekursif terakhir memiliki parameter `angka = 0` dan program selesai.

SOAL 5

Source:

```
def kombinasi(n,m):
    if m == 0 or m == n:
        return 1
    else:
        return kombinasi(n-1,m-1)+ kombinasi(n-1,m)
n=int(input("Masukkan jumlah keseluruhan: "))
m=int(input("Masukkan yang ingin dicari: "))
if n>m:
    print(kombinasi(n,m))
else:
    print("Jumlah variabel yang memenuhi terlalu banyak")
```

Output:

```
Masukkan jumlah keseluruhan: 5
Masukkan yang ingin dicari: 2
10
PS C:\Users\ASUS\Documents\File
Masukkan jumlah keseluruhan: 10
Masukkan yang ingin dicari: 5
252
```

Penjelasan:

Program berfungsi untuk menghitung kombinasi, pertama program meminta user memasukkan jumlah variabel dan jumlah variabel yang memenuhi. Yang akan dicek apakah jumlah variabel total lebih besar dibandingkan variabel yang memenuhi. Jika ya maka akan masuk kedalam fungsi, di dalam fungsi dicek apakah jumlah variabel yang memenuhi bernilai 0 karena jika ya hasilnya akan 1 atau jika nilai total variabel dan variabel yang memenuhi sama. Jika tidak ada yang memenuhi maka akan terjadi rekursif dengan mengembalikan fungsi kombinasi dengan parameter(n-1 dan m-1) ditambah fungsi kombinasi dengan parameter(n-1,m). Fungsi akan terus berulang sampai semua fungsi akan mengembalikan nilai 1 dan dijumlah sehingga kita mendapatkan kombinasi n,m.